

Fundamentos de la programación 2

Ejercicios. Hoja 2 extendida

Los ejercicios marcados como **Evaluable** tienen que subirse al CV durante la sesión de laboratorio.

- Se subirá **únicamente el archivo Program.cs**
- Sólo hará la entrega uno de los miembros del grupo.
- La línea 1 (y siguientes) deben contener los nombres de los alumnos que hacen la entrega de la forma:

```
// Nombre Apellido1 Apellido2
// Nombre Apellido1 Apellido2
```

1. Implementar los algoritmos ordenación por inserción, selección y burbuja vistos en clase. Hacer pruebas de rendimiento con vectores de tamaños dados generados aleatoriamente y sacar conclusiones. ¿Existen casos más o menos favorables para cada uno de estos algoritmos? Es decir, ¿son sensibles al orden de los elementos del vector a ordenar y funcionan mejor/peor en función de ese orden?
2. Tal como hemos visto, el algoritmo de ordenación por selección visto en clase busca el elemento mínimo y lo coloca en la primera posición, luego el siguiente mínimo entre los restantes y lo coloca en la segunda, etc. Es posible implementar una variante de este algoritmo apoyándose en la siguiente idea: en el mismo recorrido que se hace para determinar el mínimo, puede también determinarse el máximo, que podría colocarse en la última posición, luego el siguiente máximo en la penúltima. Es decir, se va ordenando por la izquierda y la derecha del vector simultáneamente.

Implementar este algoritmo de *dobleSelección*. Hacer comparativas de tiempo con el algoritmo estándar de selección y razonar doble los resultados.

3. Los algoritmos de ordenación de arrays vistos en clase son generales en el sentido de que sirven para ordenar arrays de cualquier tipo que tenga una definida una relación de orden (operación $\leq$), como los enteros. Sin embargo, para algunos problemas concretos es posible sacar provecho de las particularidades de los datos. Por ejemplo, si sabemos que el array a ordenar $v[0..n-1]$ contiene enteros en un rango *acotado y suficientemente pequeño*, es decir, $0 \leq v[i] \leq \max$ para todo $i \in \{0, \dots, n-1\}$, con $\max$ relativamente pequeño. En ese caso podemos utilizar el siguiente algoritmo:

- construir la tabla de frecuencias t asociada a v . Esta tabla no es más que un vector $t[0..\max]$ tal que $t[i]$ contiene el número de apariciones del elemento i en el array inicial v .
- a partir de esta tabla obtener la ordenación del vector v . La idea es: colocar $t[0]$ ceros al principio de v , a continuación colocar $t[1]$ unos y así sucesivamente.

Por ejemplo, consideremos

$$v = [0, 2, 1, 3, 4, 2, 3, 1, 2, 0, 3, 4]$$

Todos los elementos están en el rango $[0, 4]$ ($\max = 4$). La tabla de frecuencias asociada es:

	0	1	2	3	4
t	2	2	3	3	2

Ahora, colocamos en v en este orden 2 ceros, 2 unos, 3 doses, 3 treses y 2 cuatros, y obtenemos el array ordenado:

$$v = [0, 0, 1, 1, 2, 2, 2, 3, 3, 3, 4, 4]$$

Implementar este algoritmo con un método `void Ordena(int [] v)`, que funcione para vectores de enteros en el rango $[0, 999]$. ¿Es este algoritmo mejor que los que hemos visto en clase?

(Opcional) Ampliar la funcionalidad de modo que en tiempo de ejecución se determine el mínimo (min) y máximo (max) de los elementos del vector y el algoritmo trabaje con el rango correspondiente $[min, max]$.

4. Implementar el algoritmo de "búsqueda trinaría" para vectores ordenados, que funciona de modo similar al búsqueda binaria, pero en cada iteración, en vez de dividir el subvector de búsqueda en 2 partes, lo divide en 3. ¿Es mejor o peor que la búsqueda binaria?
5. Implementa una variante del algoritmo de ordenación por inserción utilice la búsqueda binaria para encontrar la posición correcta donde insertar cada elemento en vez de hacer la búsqueda secuencial que hace la versión original. Para ello será útil implementar un método que encuentre el índice del primer elemento que sea mayor o igual al número dado. Si no existe tal elemento, devuelve -1.

¿Es mejor o peor que el algoritmo de inserción original? Hacer algunos experimentos de medida de tiempos para comparar el rendimiento de ambos algoritmos.

6. Dado un array v ordenado de manera ascendente que ha sido rotado en un punto desconocido, escribe un método similar a la búsqueda binaria `int BuscaRot(int [] v)` que encuentre el índice donde se produce la rotación. Por ejemplo, si el array original era: $[1, 2, 3, 4, 5, 6, 7]$ y se rotó en algún punto, podría haber quedado así $[4, 5, 6, 7, 1, 2, 3]$. En este caso, la rotación se produce en el índice 4.
7. Dado un array ordenado de enteros v y un número a buscar, implementa un algoritmo `void Secuencia(int [] v, int pri, int ult)` similar a la búsqueda binaria, que encuentre el segmento del array donde se encuentra el número, es decir, el primer y último índices (`pri` y `ult`) en los que aparece dicho número en el array. Si el número no está en el array, el algoritmo debe devolver los índices -1, -1.

Por ejemplo, si el array es $[1, 2, 2, 2, 3, 4, 5]$ y el número a buscar es 2, el algoritmo debe devolver $[1, 3]$.

8. (**Evaluable**) Dada una matriz bidimensional `mat` de enteros ordenada, donde cada fila está ordenada de izquierda a derecha y la primera columna de cada fila es mayor o igual que el último elemento de la fila anterior, implementa un método `(int,int) Busca(int [,] mat, int e)` que busque un número objetivo con complejidad similar a la búsqueda binaria.

Puede ser útil implementar métodos que conviertan posiciones en la matriz (bidimensionales) a posiciones en el array unidimensional correspondiente y viceversa. Por ejemplo, para una matriz de tamaño 3×4 , la posición $(2, 3)$ corresponde al índice 11 ($2 * 4 + 3 = 11$) en el array unidimensional correspondiente.